

arc42 Architekturdokumentation

Thesis-Jensclicker Projekt

Version 1.0 - Februar 2026
Mithilfe von KI

1. Einführung und Ziele

1.1 Aufgabenstellung

Das Thesis-Jensclicker Projekt ist eine modulare Spring Boot Anwendung zur Verwaltung von Abschlussarbeitsthemen an der Heinrich-Heine-Universität Düsseldorf. Die Anwendung ermöglicht es Dozenten, ihre Profile und Forschungsthemen zu präsentieren, und Studierenden, passende Themen für Abschlussarbeiten zu finden.

Die Architektur folgt einem Multi-Modul-Ansatz mit Gradle Submodulen:

- Profil-Modul: Verwaltung von Dozentenprofilen und Forschungsthemen
- Homepage-Modul: Einstiegsseiten und Navigation
- Utility-Modul: Gemeinsam genutzte Hilfsfunktionen

1.2 Qualitätsziele

Priorität	Qualitätsziel	Szenario
1	Modularität und Wartbarkeit	Gradle Submodule ermöglichen unabhängige Entwicklung
2	Sicherheit	OAuth2 mit GitHub für sichere Authentifizierung
3	Datenpersistenz	PostgreSQL mit Flyway für zuverlässige Datenhaltung

1.3 Stakeholder

Rolle	Kontakt	Erwartungshaltung
Dozenten	TEACHER-Rolle	Profile verwalten, Themen anbieten
Studierende	STUDI-Rolle	Themen finden und durchsuchen
Administratoren	ADMIN-Rolle	Benutzerverwaltung, Rollen zuweisen

2. Randbedingungen

2.1 Technische Randbedingungen

- Java 21 mit Spring Boot 3.5.7
- Gradle als Build-Tool mit Multi-Modul-Projekt
- PostgreSQL als Datenbank
- Flyway für Datenbankmigrationen
- Thymeleaf als Template Engine
- Docker Compose für lokale Entwicklung

2.2 Organisatorische Randbedingungen

- Entwicklung im Rahmen einer Programmierpraktikum-Abschlussarbeit
- Teamarbeit mit klarer Modulaufteilung
- Abgabetermin: Februar 2026

3. Kontextabgrenzung

3.1 Fachlicher Kontext

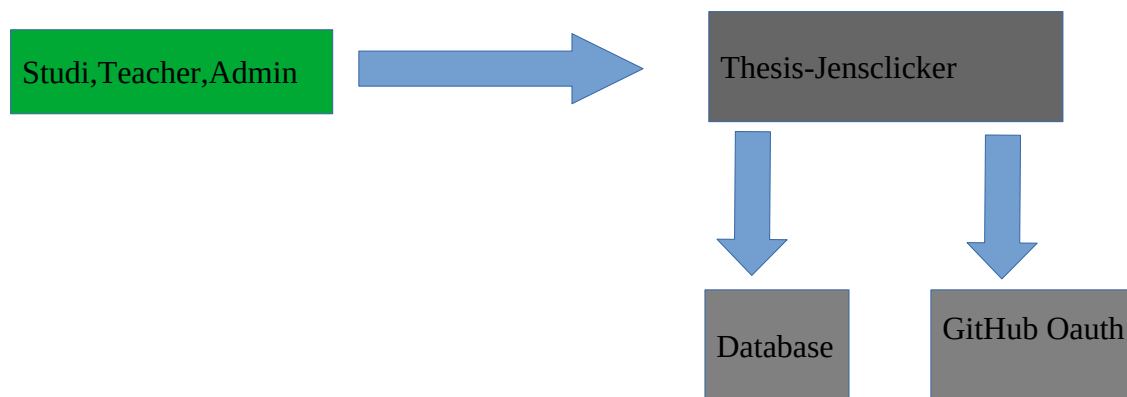
Das System interagiert mit folgenden externen Akteuren und Systemen:

Nachbarsystem/Akteur	Beschreibung
Studierende	Suchen und finden Themen für Abschlussarbeiten
Dozenten	Erstellen und verwalten Profile sowie Forschungsthemen
Administratoren	Verwalten Benutzerrollen und System-Einstellungen
GitHub OAuth	Authentifizierung der Benutzer über GitHub-Konten
PostgreSQL	Persistierung von Profilen, Themen, Links und Dateien

Kontextabgrenzungsdiagramm

Das folgende Diagramm zeigt die Kontextabgrenzung des Systems:

[Diagramm: Studierende, Dozenten und Administratoren nutzen das Thesis-Jensclicker System, welches mit GitHub OAuth (Authentifizierung) und PostgreSQL (Datenhaltung) kommuniziert]



3.2 Technischer Kontext

Die Anwendung kommuniziert über folgende technische Schnittstellen:

- HTTP/HTTPS für Webanfragen (Port 8080)
- JDBC für Datenbankzugriff zu PostgreSQL
- OAuth2-Protokoll für Authentifizierung mit GitHub
- Flyway für automatische Schema-Migrationen

5. Bausteinsicht

5.1 Ebene 1: Gesamtsystem (Whitebox)

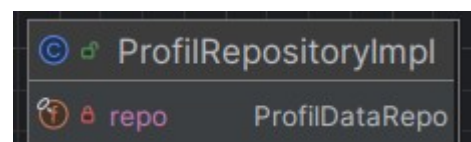
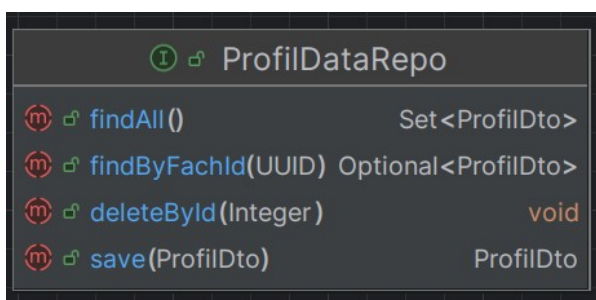
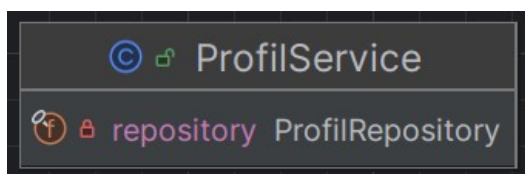
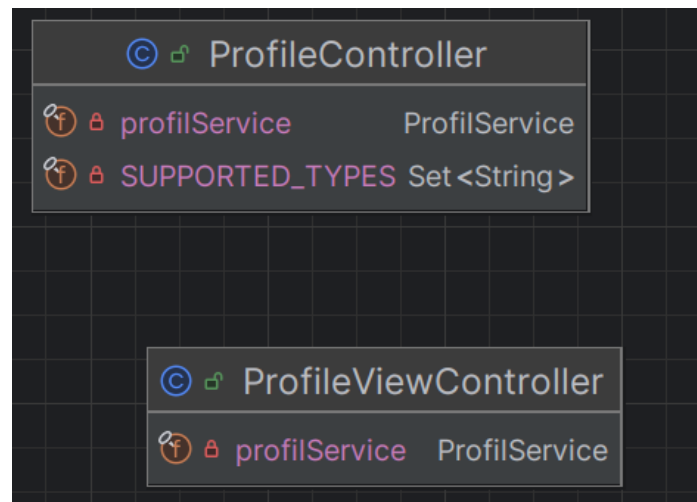
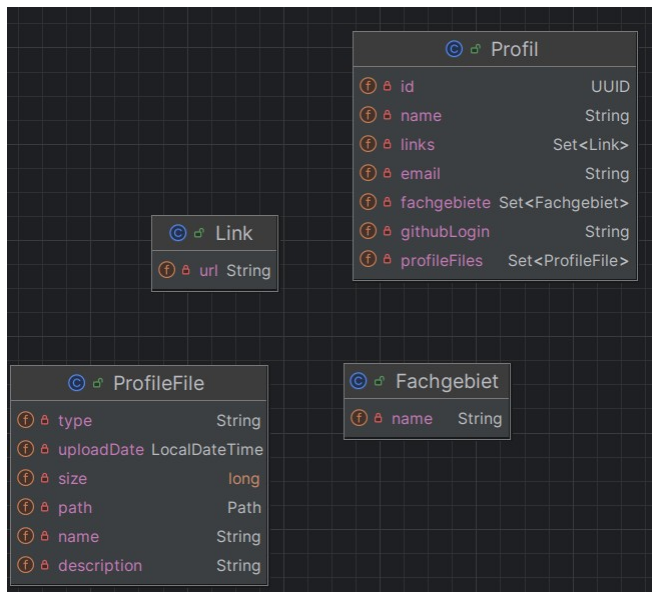
Das Gesamtsystem besteht aus drei Hauptmodulen, die über Gradle als Submodule organisiert sind:

Baustein: Profil-Modul

Verantwortlichkeit: Verwaltung von Dozentenprofilen und Forschungsthemen

Schnittstellen:

- REST-Controller für Profilbearbeitung und Datei-Upload
- View-Controller für Thymeleaf-Templates
- Repository-Interfaces für Datenbankzugriff

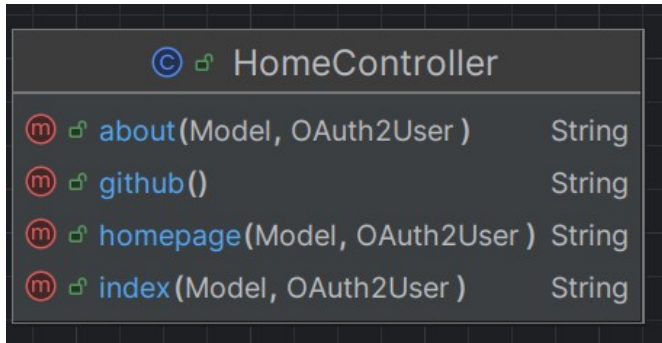


Baustein: Homepage-Modul

Verantwortlichkeit: Bereitstellung der Einstiegsseiten und Navigation

Schnittstellen:

- HomeController für Hauptseiten (/ , /teacher, /about)
- OAuth2-Redirect-Handling (/gh-login)
- Integration mit NavMapper für rollenbasierte Navigation

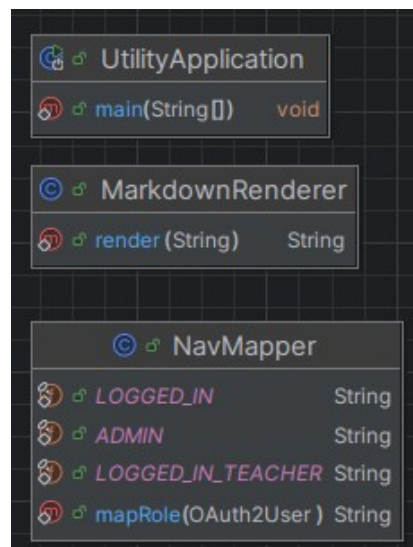
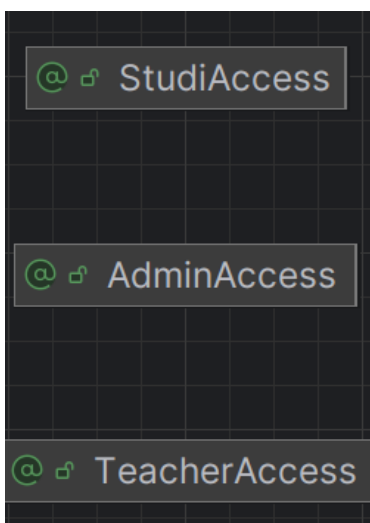


Baustein: Utility-Modul

Verantwortlichkeit: Gemeinsam genutzte Hilfsfunktionen

Schnittstellen:

- NavMapper für Navigation-Mapping
- MarkdownRenderer für Markdown-Rendering
- Security-Annotations (@AdminAccess, @TeacherAccess, @StudiAccess)



5.2 Ebene 2: Profil-Modul (Whitebox)

Das Profil-Modul folgt einer klaren Schichtenarchitektur mit folgenden Ebenen:

Domain Layer

Klasse

Profil

Thema

Fachgebiet

Link

ProfileFile

Verantwortlichkeit

Aggregat-Root: Verwaltet Name, E-Mail, GitHub-Login, Links, Fachgebiete und Dateien

Abschlussarbeitsthema mit Titel, Beschreibung, Voraussetzungen und CS-Modulen

Forschungsbereich eines Profils (z. B. KI, Datenbanken)

Externe URL (z. B. persönliche Website, Publikationen)

Hochgeladene Datei (Name, Typ, Größe,

Voraussetzung

Binärdaten)
Anforderung für ein Thema (z. B. erforderliche
Module)

Application Layer

ProfilService: CRUD-Operationen für Profile, Verwaltung von Links, Fachgebieten und Dateien.
Bietet Business-Logik wie existsProfileByGithubLogin().

ThemaService: Verwaltung von Abschlussarbeitsthemen mit Voraussetzungen und
Modulzuordnungen.

Persistence Layer

Implementiert die Repository-Interfaces mit Spring Data JDBC. Nutzt DTOs für die
Datenbankabbildung und separate DAOs (ProfilDataRepo, ThemaDataRepo) für CRUD-
Operationen.

Web Layer

ProfileController: REST-Endpunkte für Profilbearbeitung und Datei-Upload
ProfileViewController: Rendering von Profil-Ansichten mit Thymeleaf

8. Querschnittliche Konzepte

8.1 Sicherheit

Die Anwendung nutzt Spring Security mit OAuth2:

- Authentifizierung über GitHub OAuth2
- Rollenbasierte Autorisierung (ADMIN, TEACHER, STUDI)
- Method-Security mit Custom Annotations (@AdminAccess, @TeacherAccess, @StudiAccess)
- HTML-Sanitisierung mit OWASP Java HTML Sanitizer für Benutzereingaben

8.2 Persistenz

Spring Data JDBC mit PostgreSQL:

- Aggregate Roots mit @PersistenceCreator
- DTOs für Datenbankabbildung (ProfilDto, ThemaDto, etc.)
- Flyway für Versionskontrolle der Datenbankschemas
- Repository-Pattern mit Interface-Segregation

8.3 Exception Handling

Custom Exceptions im Profil-Modul:

- FileNotFoundException: Datei nicht gefunden
- FileTooLargeException: Datei überschreitet Größenlimit
- UnsupportedFileTypeException: Nicht unterstützter Dateityp
- NichtVorhandenException: Ressource nicht vorhanden
- UngueltigeEingabeException: Validierungsfehler

8.4 Validierung

Jakarta Bean Validation mit Annotations:

- @NotBlank für erforderliche Felder
- @Email für E-Mail-Validierung
- Custom Validierung in Services

9. Architekturentscheidungen

9.1 Gradle Multi-Modul-Projekt

Entscheidung: Aufteilung in separate Gradle-Submodule (Profil, Homepage, Utility)

Begründung:

- Klare Trennung der Verantwortlichkeiten
- Parallele Entwicklung durch Teammitglieder möglich
- Wiederverwendbarkeit des Utility-Moduls
- Bessere Testbarkeit durch Modulgrenzen

9.2 Spring Data JDBC statt JPA

Entscheidung: Verwendung von Spring Data JDBC anstelle von JPA/Hibernate

Begründung:

- Einfacheres Modell ohne implizite Lazy Loading-Probleme
- Klarere Aggregat-Grenzen
- Bessere Performance-Kontrolle
- Weniger Magic und explizitere Datenbank-Operationen

9.3 OAuth2 mit GitHub

Entscheidung: GitHub OAuth2 als einziger Authentifizierungsmechanismus

Begründung:

- Alle Studierenden und Dozenten der HHU haben GitHub-Accounts
- Keine eigene Benutzerverwaltung notwendig
- Single Sign-On Erfahrung
- Sicherheit durch etablierte OAuth2-Standards

10. Qualitätsanforderungen

10.1 Testabdeckung

Das Projekt enthält Tests auf mehreren Ebenen:

- WebMvcTest für Controller (@WebMvcTest mit MockMvc)
- DataJdbcTest für Repository-Layer mit Testcontainers
- ArchUnit-Tests für Architekturregeln (ProfilArchUnitTest)
- Custom Security Context Factory für OAuth2-Tests (WithMockOAuth2User)
- TestApplication für Integration Tests

10.2 Code-Qualität

- ArchUnit-Tests erzwingen Schichtenarchitektur
- Custom Rules für Getter/Setter-Konventionen
- Dokumentierte Verwendung von KI-Tools (ki.txt)
- SpotBugs-Annotations für statische Code-Analyse

11. Risiken und technische Schulden

11.1 Bekannte TODOs und FIXMEs

Im Code wurden folgende Punkte als verbesserungswürdig markiert:

- HomeController: if-statement mit principal-Check könnte durch bessere Security-Config ersetzt werden
- ProfilService.getProfileByGithubLogin(): Funktioniert nur wenn github_login unique ist - erste Übereinstimmung wird zurückgegeben

11.2 Identifizierte Risiken

- GitHub Login ist nicht eindeutig: Mehrere Profile könnten denselben GitHub-Login haben
- Datei-Upload ohne Virus-Scanning
- Keine Backup-Strategie für hochgeladene Dateien dokumentiert
- Testcontainers benötigen Docker - alternative H2-Konfiguration für CI/CD empfohlen

12. Glossar

Begriff

Aggregat

DTO

DAO

OAuth2

Flyway

Thymeleaf

ProPra

Testcontainers

ArchUnit

Definition

Domain-Driven Design Konzept: Gruppe von Objekten mit einer Root-Entity

Data Transfer Object: Objekt für Datentransfer zwischen Schichten

Data Access Object: Interface für Datenbankoperationen

Open Authorization: Standard für Zugriffsberechtigungen

Tool für Datenbankmigration und Versionskontrolle von Schemas

Java Template Engine für Server-seitiges HTML-Rendering

Programmierpraktikum: Lehrveranstaltung an der HHU

Java-Bibliothek für Docker-basierte Integration Tests

Bibliothek zum Testen von Architekturregeln